

Author

Ankit Kumar

Roll No.: 23F30004032

Email: 23f30004032@ds.study.iitm.ac.in

Diploma, BS in Data Science & Applications, IIT Madras

Founder, Zyrodev.com

Description

Vehicle Parking App V2 is a web application to manage multiple parking lots and 4-wheeler spots. An admin can create and configure parking lots, and users can register, book the first available parking spot, and release it. The system records timestamps, calculates parking costs, and automates daily reminders, monthly reports, and CSV exports.

AI/LLM Usage

No AI or LLM tools were used in implementing core functionality.

Technologies Used

- **Flask:** REST API framework
- **Flask-JWT-Extended:** Token-based authentication and role management
- **SQLite & SQLAlchemy:** Lightweight database and ORM for data modeling
- **Redis:** Caching for performance
- **Celery:** Asynchronous batch jobs and scheduled tasks
- **Vue.js:** Front-end framework for reactive UI
- **Bootstrap 5:** Responsive styling and layout
- **Axios:** HTTP client for API calls
- **Pandas:** CSV export processing

DB Schema Design

1. **User** (id, email, fullname, password_hash, role)
2. **Lot** (id, name, address, price_per_hour, total_spots)
3. **Spot** (id, lot_id → Lot, spot_number, status 'A'/'O')
4. **Reservation** (id, user_id → User, spot_id → Spot, in_time, out_time, cost)

Constraints:

- Deleting a lot requires no occupied spots.

- Enum fields enforce valid roles and statuses.
- Foreign-key indexes for efficient joins.

API Design

- - Auth
 - **POST /auth/register:** User sign-up
 - **POST /auth/login:** JWT token retrieval
- - Admin Endpoints (JWT 'admin' role)
 - **POST /admin/lots:** Create parking lot
 - **GET /admin/lots:** List lots
 - **PUT /admin/lots/{id}:** Edit lot
 - **DELETE /admin/lots/{id}:** Delete empty lot
 - **GET /admin/spots/{lot_id}/status:** Spot availability
 - **GET /admin/users:** List all users
- - User Endpoints (JWT 'user' role)
 - **GET /user/lots:** Available lots
 - **POST /user/reserve:** Book first-free spot
 - **POST /user/release/{reservation_id}:** Release spot, calculate cost
 - **GET /user/report/monthly:** Trigger monthly HTML report
 - **GET /user/export/csv:** Trigger CSV export of reservations

Architecture and Features

The backend follows a modular Flask structure:

- app.py initializes Flask, extensions (db, cache, jwt), and Celery.
- models.py contains SQLAlchemy ORM definitions.
- controllers/ folders organize auth, admin, and user routes.
- tasks.py defines Celery jobs for reminders, reports, and exports.

The frontend is a Vue.js CLI project under frontend/, using Bootstrap for layout.

Components include login forms, dashboards, and settings. Axios manages API calls with JWT in localStorage.

Key features:

- Role-based login and access control
- Automated allocation of the first available spot
- Real-time caching of lot data

- Scheduled Celery tasks for daily reminders and monthly reports
- User-triggered async CSV export with email notification

Video

Link:  MAD 2 Project 23F3004032.mp4